PYNQ 接口检查

把 Python 应用移到另一块板时，同一种 IP 可能换了实例名、层次路径或基地址。应用真正依赖的是哪一种 IP、哪一版寄存器表，以及程序要访问哪些偏移。本技能把这些要求写成一份接口契约，先找出匹配实例，再决定是否加载 Overlay。它适用于向处理器公开寄存器的 IP，示例用 AXI GPIO。

这条方法来自原工程的两条 **DMA** 通路。 实际导出的 HWH 中，dma_pcm 和 dma_spectrum 都是 xilinx.com:ip:axi_dma:7.1，两者的 S2MM_DMACR 和 S2MM_DMASR 偏移也同为 0x30、0x34，但一个搬运时域样本，另一个搬运频谱。仅按类型取第一项无法区分两条通路。因此，工具遇到同类型多实例时要求显式选择完整路径。可在配套工程包的 src/bd/create_pre_adc_bd.tcl 中核对两条连接。

先在普通电脑上试用

进入本说明所在目录。离线方式只依赖 Python 标准库。

```
python check_overlay.py examples/gpio_contract.json --metadata
examples/gpio_metadata.json
python selftest.py
```

第一条从示例元数据中找到 panel/buttons，成功输出中的 scope 为 offline-metadata，register_offsets 列出两个寄存器的字节偏移。第二条运行本技能的自测。示例数据供练习，板卡型号和物理地址均无需写入契约。

契约字段	**GPIO** 示例	如何确定
vlnv	xilinx.com:ip:axi_gpio:2.0	从软件所支持的 IP 规格选取厂商、库、名称和版本，要求完整匹配
registers	{"GPIO_DATA": 0, "GPIO_TRI": 4}	从该版 IP 的寄存器表填写软件实际使用的寄存器名与字节偏移，偏移相对该 IP 的起始地址

初次接入可做一次负对照。复制契约到 wrong_contract.json，仅把 GPIO_TRI 改成 8，用副本重跑第一条命令。预期退出码为 1；换回原契约应为 0。如果两次都成功，先检查传入的是否是不同文件。

在兼容板卡上操作

1. 从能运行 PYNQ 的板卡镜像启动。在同一次 Vivado 导出中取出为这块板生成的全量 .bit 与配套 .hwh，同目录、同主文件名放置，例如 design.bit 和 design.hwh。同名只是查找条件，文件是否来自同一设计仍需由导出者确认。

2. 在板上 Python 环境执行下列第一条命令。工具以 Overlay(..., download=False) 解析元数据，从 ip_dict 匹配接口。成功只表示文件描述的接口满足契约。若有多个相同 VLNV 的实例，在 Vivado 设计中确认所需外设的连接，再按第二条命令指定其实际层次路径。

```
python check_overlay.py examples/gpio_contract.json --bit design.bit
```

```
python check_overlay.py examples/gpio_contract.json --bit design.bit --ip
panel/buttons
```

3. 确定设计和引脚适合该板，并停止依赖当前 PL 配置的应用。若需实际编程，在前一步成功的命令后加 --download。脚本先重复检查契约，再调用 download()，最后要求 PYNQ 的 is_loaded() 返回真。重新配置会替换当前逻辑，还可能调整 PL 时钟。

4. 做应用功能试验并分别记录板型、PYNQ 版本、设计版本、命令、实例名和观察结果。例如 GPIO 连接按钮时，先从设计图确认通道、输入方向和有效电平，再用应用驱动分别读取松开与按下时的输入位。此步骤由使用者按目标设计实施，本脚本不自动访问外设寄存器。PYNQ 官方 Overlay 使用说明 · PYNQ 加载状态的实现

换题目与换板

• 同一题目换板卡。 为新板重新生成配套设计文件。若保留相同 IP 版本、寄存器名和偏移，沿用旧契约。基地址和实例名可以变化；多实例情况下重新选择 --ip。

• 换题目或 **IP**。 新建契约，例如将 GPIO 替换为自定义计数器，从计数器的接口规格填写 VLNV、状态寄存器名称和偏移。契约的预期值应独立于待测的 HWH。然后换用新设计的文件，依次进行负对照、元数据检查、加载与功能试验。

• 带回电脑复查。 离线 JSON 采用 Overlay.ip_dict 所用的字段结构。顶层键为完整实例名，实例对象中的 type 为 VLNV，registers 下各寄存器的 address_offset 为相对偏移。只需保存这些字段，示例文件给出了完整格式。离线快照应注明来源，手工填写的示例只能用于练习。

验证范围和失效条件

在本目录可重放参赛工程的两个 DMA 实例。下例读取从 Vivado HWH 提取的寄存器和地址字段；期望偏移取自 AXI DMA 的 S2MM 控制、状态寄存器布局。第一次运行会因两个实例均匹配而返回 1；后两次分别返回 0，基地址为 0x40400000、0x40410000，两者的寄存器偏移均为 0x30、0x34。这些命令只读取 JSON。

```
python check_overlay.py examples/ax7021_dma_contract.json --metadata
examples/ax7021_dma_metadata.json

python check_overlay.py examples/ax7021_dma_contract.json --metadata
examples/ax7021_dma_metadata.json --ip dma_pcm

python check_overlay.py examples/ax7021_dma_contract.json --metadata
examples/ax7021_dma_metadata.json --ip dma_spectrum
```

环节	本次覆盖	能回答的问题
AX7021 工程导出的 HWH 字段	上述三个离线回放均符合预期	同一接口要求能否用于不同地址的实例，并在选择不明确时停止
离线契约与失败用例	10 项命令行用例通过，涵盖更换设计、移动实例与基地址、同类型多实例、错误偏移等	能否选对 IP、检查必需寄存器并拒绝歧义或错误布局

环节	本次覆盖	能回答的问题
板端调用顺序的离线测试	另 5 项用例以软件替身模拟 PYNQ 接口，检查缺少 HWH、默认不下载、接口不符时拒绝下载、显式下载及加载状态失败	脚本是否在预期时刻调用 PYNQ 方法，两个控制路径是否能区分 pynq-metadata 和 pynq-loaded
PYNQ 实板解析与下载	本技能未取得兼容板上的实测记录	需在目标板实际执行上述命令，记录 PYNQ 是否成功识别接口并报告已加载
外设工作状态	需按每个题目设计功能试验	时钟、复位、引脚、外接信号与寄存器行为是否符合设计

原作品在 AX7021 上完成的是 FreeRTOS 固件下的采集、频带显示与 SD 回读。该实测覆盖原作品数据链；上表单独列出 PYNQ 技能的验证范围。

找不到指定 VLNV 时，检查 IP 版本与 HWH 来源。多个实例匹配时指定完整路径。寄存器缺失或偏移不符时，对照 IP 规格与 Vivado 地址映射，重新导出正确的 HWH；随意修改契约去迎合输入会掩盖接口变更。命令返回 0 表示所选阶段成功，非零时先阅读诊断并停止后续步骤。

本流程面向支持本板全量 .bit 和 .hwh 的 PYNQ 环境。纯流接口、依赖部分重配置或仅使用 XRT 容器的设计超出此示例范围。寄存器偏移匹配不能确认位宽、访问权限、读写副作用、DMA 传输或外设功能。板卡型号、器件和引脚约束仍需在生成位流时正确选择。